

North Bengal International University, Rajshahi.



LAB REPORT

Course Title: Neural Network & Fuzzy System Sessional.

Course Code: CSE 0613-4222

Experiment No: 03

Name of Experiment: CNN Feature Learning Analysis Using
Feature Map Visualization.

SUBMITTED BY:

MD. RAZAB ALI
ID: 0712340012101011
DEPARTMENT OF CSE
7TH SEMESTER
BATCH-26TH
NBIU, RAJSHAHI.

SUBMITTED TO:

SAKURAH ISMAIL
LECTURER
DEPARTMENT OF CSE
NBIU, RAJSHAHI.

SUBMISSION DATE: 03 AUG 2026

Name of Experiment: CNN Feature Learning Analysis Using Feature Map Visualization

Theory: Convolutional Neural Networks (CNNs) are a specialized class of deep neural networks designed to process data with a grid-like topology, most notably images. Unlike traditional Fully Connected Neural Networks (FCNNs), which flatten input data and lose spatial relationships, CNNs utilize a mathematical operation called convolution to preserve the spatial hierarchy of features. This architectural choice makes them exceptionally efficient at recognizing patterns, ranging from simple edges to complex objects.

Code:

```
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)
from tensorflow.keras.applications.vgg16 import VGG16
from tensorflow.keras.applications.vgg16 import preprocess_input
from tensorflow.keras.preprocessing.image import load_img
from tensorflow.keras.preprocessing.image import img_to_array
from tensorflow.keras.models import Model
from matplotlib import pyplot
from numpy import expand_dims
from matplotlib import pyplot

import warnings
warnings.filterwarnings('ignore')

#Load the model
model = VGG16()
model2 = VGG16()
# Summary of the model
model.summary()

for layer in model.layers:

    if 'conv' not in layer.name:
        continue
    filters , bias = layer.get_weights()
    print(layer.name , filters.shape)

filters , bias = model.layers[1].get_weights()

# normalize filter values to 0-1 so we can visualize them
f_min, f_max = filters.min(), filters.max()
filters = (filters - f_min) / (f_max - f_min)

n_filters = 6
ix=1
fig = pyplot.figure(figsize=(20,15))
for i in range(n_filters):
    # get the filters
    f = filters[:, :, :, i]
    for j in range(3):
        # subplot for 6 filters and 3 channels
        pyplot.subplot(n_filters,3,ix)
        pyplot.imshow(f[:, :, j] , cmap='gray')
        ix+=1
    #plot the filters
    pyplot.show()
```

Output & Results:

Subplot for 6 filters and 3 channels

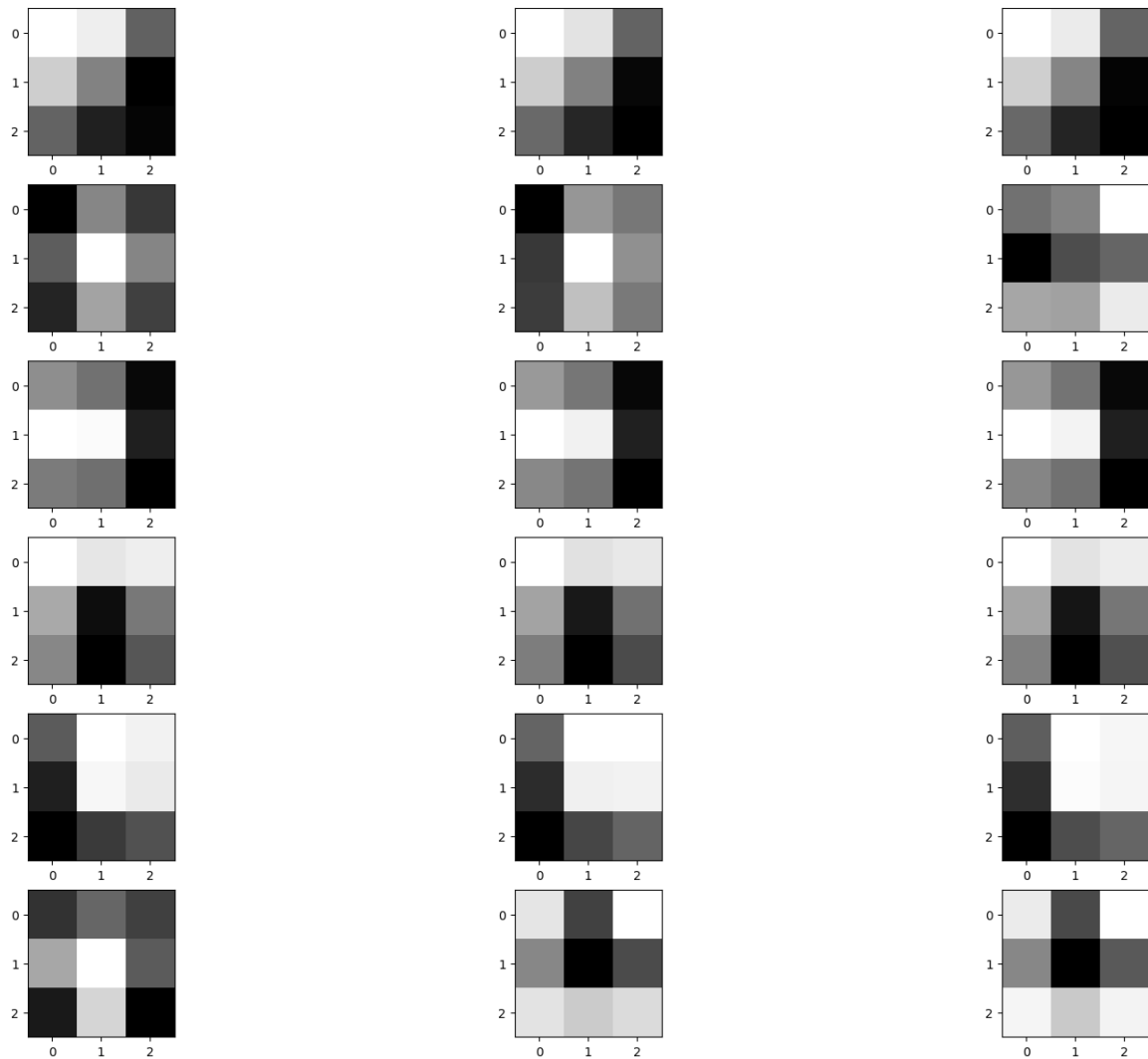


Fig. 3.1: subplot for 6 filters and 3 channels

```
for i in range(len(model.layers)):
    layer = model.layers[i]
    if 'conv' not in layer.name:
        continue
    print(i, layer.name, layer.output.shape)

model = Model(inputs=model.inputs, outputs=model.layers[1].output)
image = load_img("360_F.jpg", target_size=(224,224))
image = img_to_array(image)
image = expand_dims(image, axis=0)
image = preprocess_input(image)
features = model.predict(image)

fig = pyplot.figure(figsize=(20,15))
for i in range(1, features.shape[3]+1):
    pyplot.subplot(8,8,i)
    pyplot.imshow(features[0,:,:,i-1], cmap='gray')

pyplot.show()
```

Output & Results:
Calculating features map



Fig. 3.2: Calculating features map

Layer	Observation	Learned Features
Conv1 (Block 1)	Feature maps clearly show image boundaries and simple patterns.	Detects edges, lines, and basic features.
Conv2 (Block 2)	Feature maps become more abstract and highlight texture patterns.	Learns corners, textures, and local structures.
Conv3 (Block 3)	Feature maps focus on important object regions while removing unnecessary background.	Learns shapes, object parts, and high-level features.

Table. 3.1: Comparesn Between Learned Feature & Observation

```

model2 = VGG16()

blocks = [ 2, 5, 9, 13, 17]
outputs = [model2.layers[i].output for i in blocks]

model2 = Model( inputs= model2.inputs, outputs = outputs)

feature_map = model2.predict(image)

for i,fmap in zip(blocks,feature_map):
fig = pyplot.figure(figsize=(20,15))
#https://stackoverflow.com/a/12444777
fig.suptitle("BLOCK_{}".format(i) , fontsize=20)
for i in range(1,features.shape[3]+1):

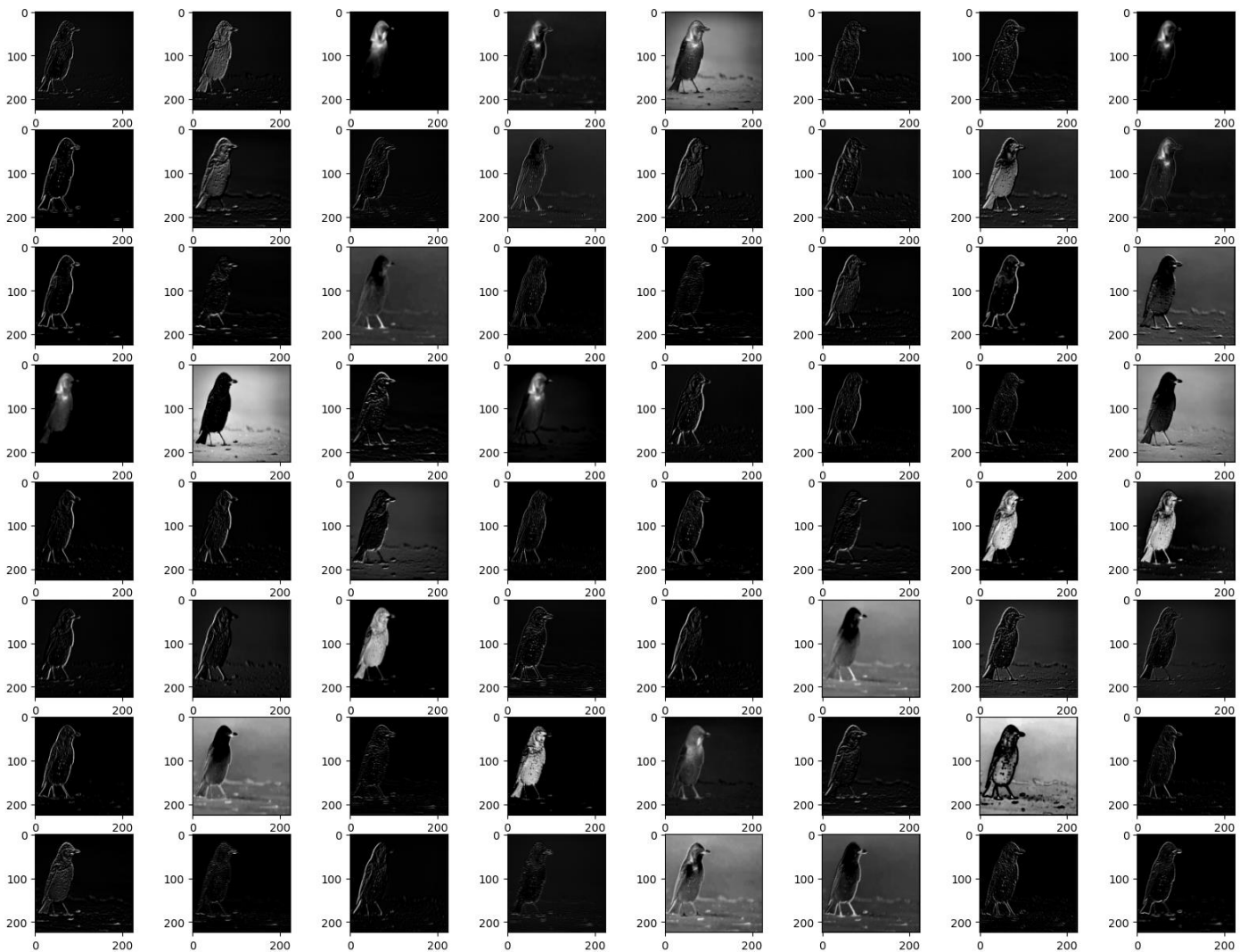
pyplot.subplot(8,8,i)
pyplot.imshow(fmap[0,:,:,:i-1] , cmap='gray')

pyplot.show()

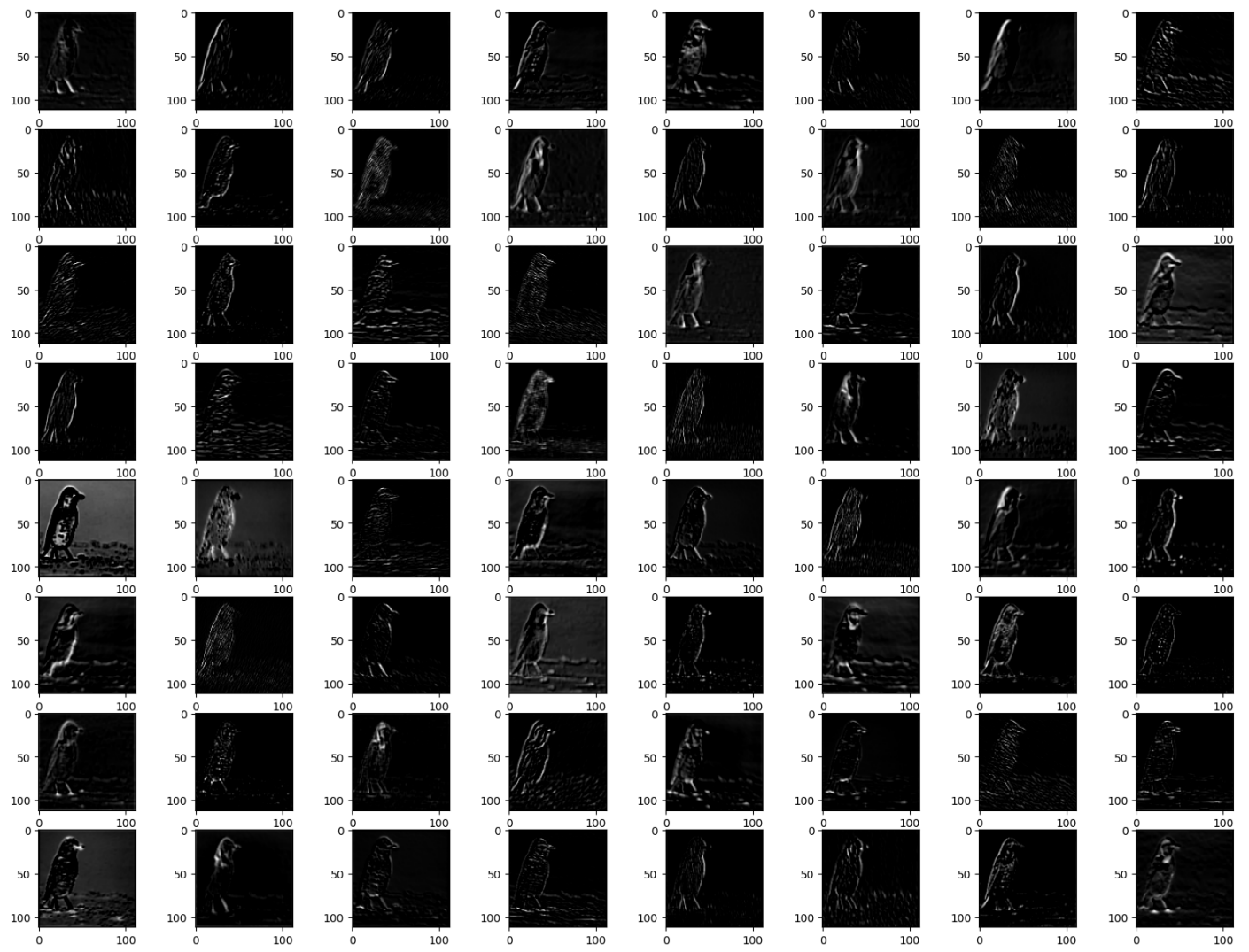
```

Output & Results:

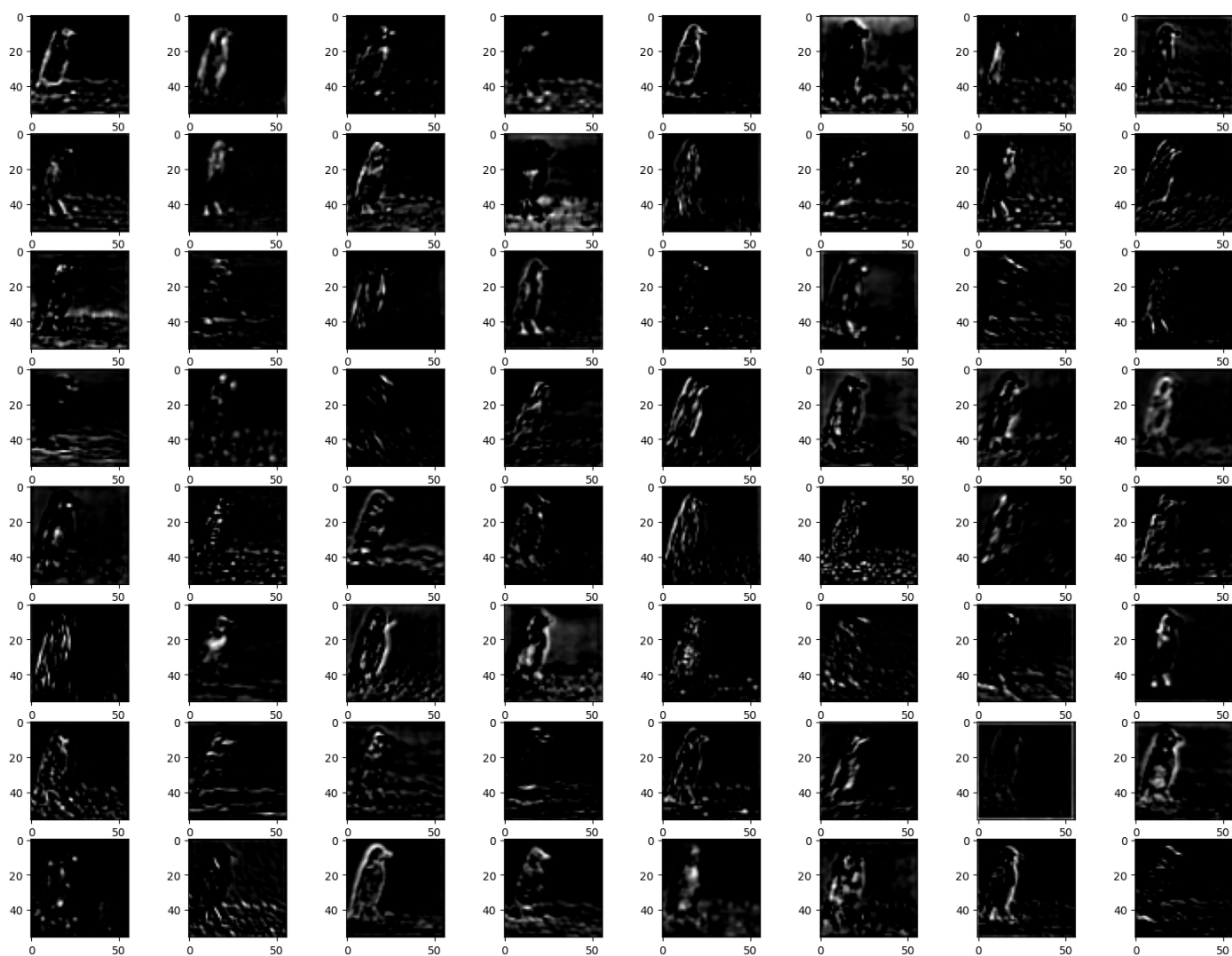
Block_2



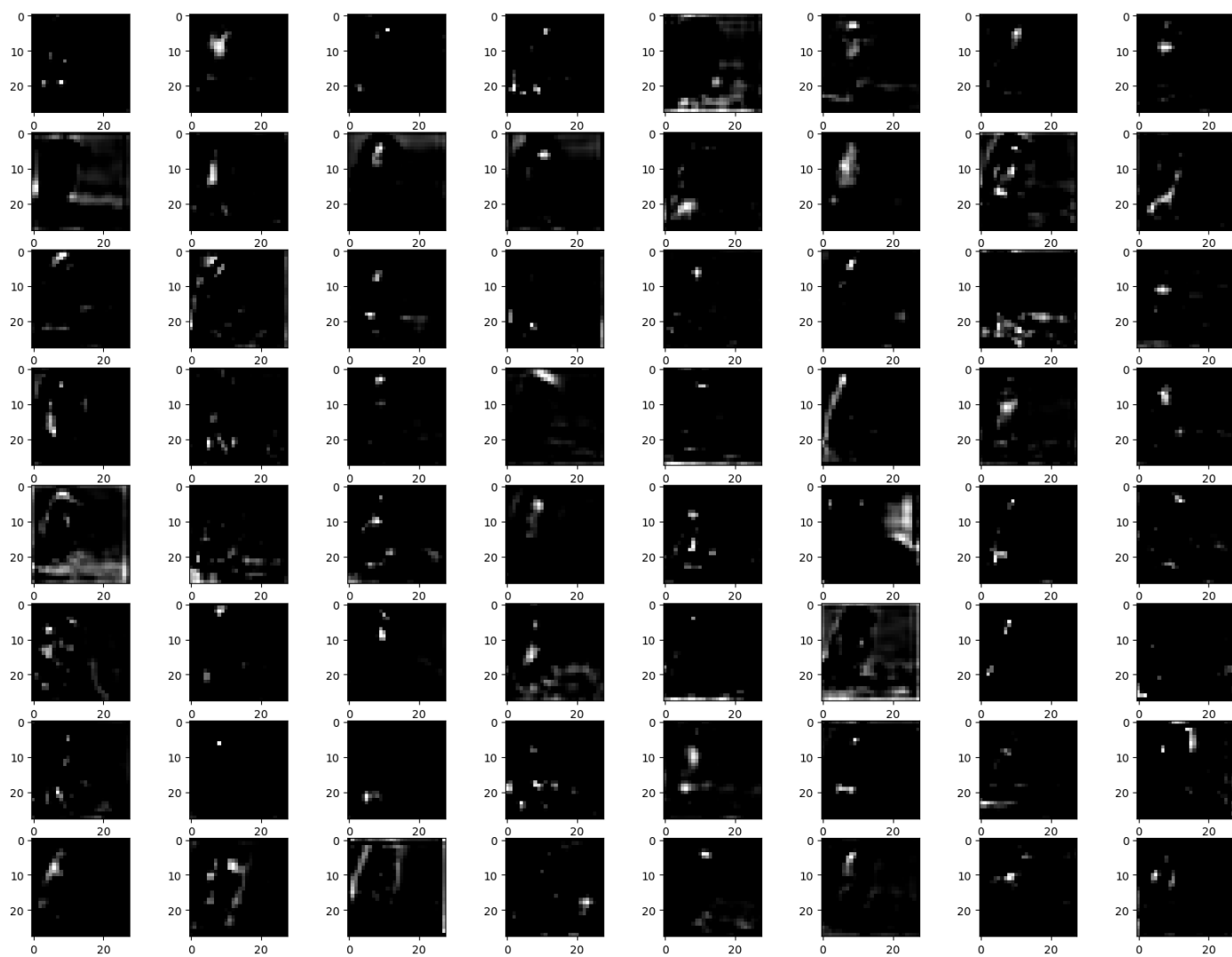
Block_5



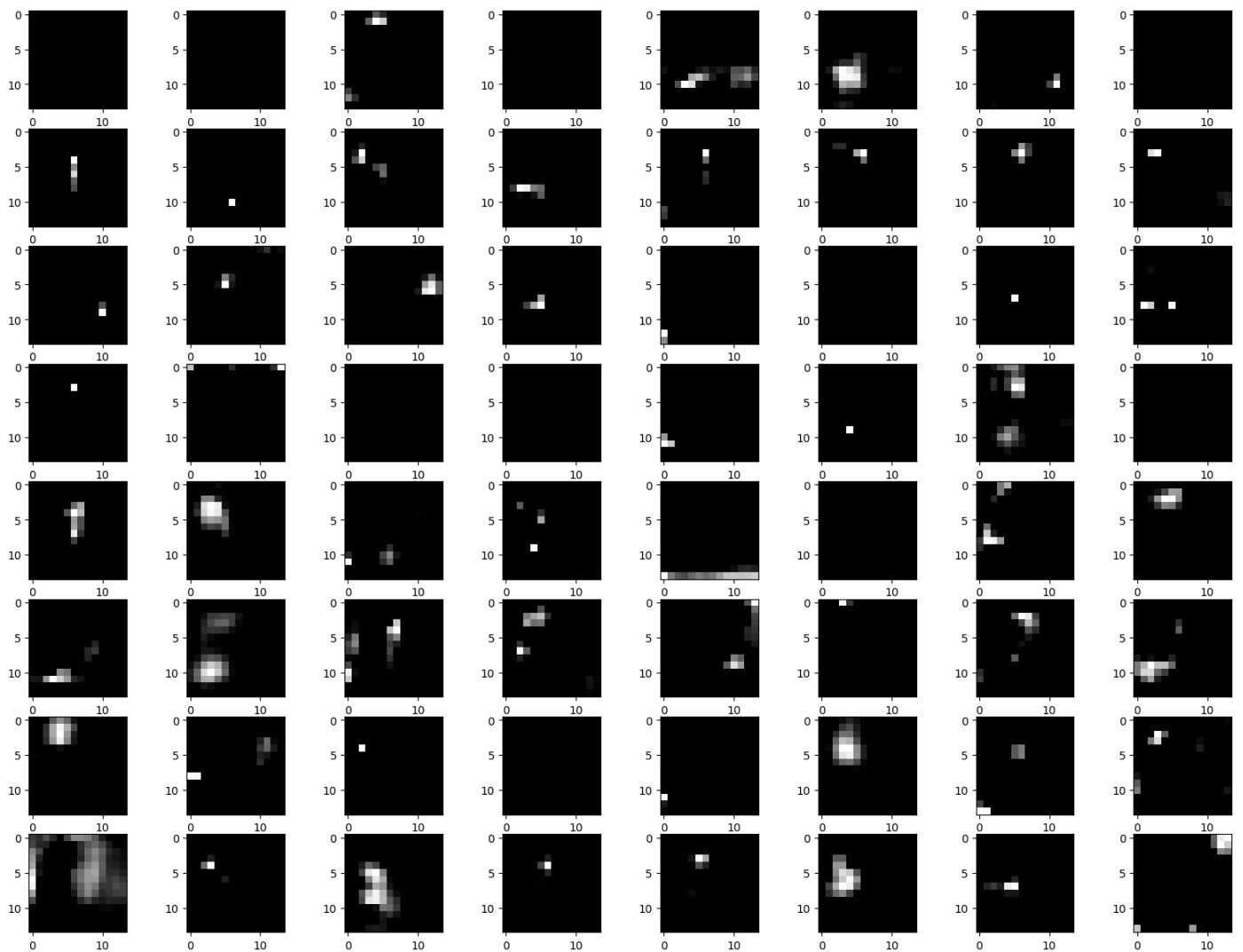
Block_9



Block_13



Block_17



After running the code and looking at the visualizations, it was really interesting to see how the model "thinks." Here's my breakdown of what's happening in each layer:

- **Which layer captures edges?**

The first few layers (like **Block 1 Conv 1**) are definitely where the edges are captured. When I looked at the feature maps for this layer, they looked almost like a sketch of the original cat image. The filters here are just looking for simple things, like where a dark area meets a light area, which helps the model outline the basic silhouette and sharp boundaries of the object.

- **Which layer captures textures?**

As we move into the middle layers (around **Block 2** and **Block 3**), the model starts picking up on textures. Instead of just seeing a single line for an edge, these layers start to react to repeating patterns, like the fur of the cat or the grainy background. It's like the model is starting to understand what the "surface" of the object looks like rather than just its outline.

- **Which layer captures shapes?**

By the time we get to the deeper middle layers (like **Block 3** and **Block 4**), the activations start looking more like shapes and fragments. You can start to see little curves that might be an ear, or a round shape that could be part of a paw. The model is essentially taking those edges and textures from earlier and piecing them together into recognizable geometric parts.

- **Which layer captures object-specific information?**

The final convolutional layers (like **Block 5**) are where the most object-specific information is stored. Honestly, to my eyes, these feature maps look pretty messy and abstract, they're mostly just bright dots or blobs. But for the AI, these blobs represent high-level concepts like "*this is a cat's face*" or "*this is a tail.*" This is the point where the model stops seeing pixels and starts seeing "*a cat.*"

It's pretty cool how the learning process is so organized. It's not just random; it's a step-by-step hierarchy:

- **First Convolution Layer (Edges & Lines):** The network starts with the basics. It acts like a simple artist drawing the first few lines of a sketch. It doesn't know what it's looking at yet; it just sees lines and boundaries.
- **Middle Layers (Corners & Textures):** Here, the network gets more sophisticated. It combines those lines to find corners and starts recognizing textures like fur, skin, or fabric. It's building up a "*vocabulary*" of patterns.
- **Deep Layers (Object Parts):** Now the model is getting smart. It uses the textures and corners to identify actual parts of an object. It's not just a texture anymore; it's a nose, an ear, or a wheel.
- **Final Layers (Complete Representation):** Finally, the model puts all those parts together to understand the whole picture. It creates a complete representation of the object, which is what allows it to say, "Hey, that's a cat!" with such high confidence.

Seeing this visualization really helped me understand why CNNs are so much better than basic networks for images, they actually "build" their understanding just like we do!

Conclusion: A **Convolutional Neural Network (CNN)** learns image features in a hierarchical manner. As the input image passes through successive convolutional layers, the network gradually extracts more complex and meaningful features.

- **First Convolution Layer (Edges & Lines):** Detects edges, simple lines, and boundaries that form the basic structure of the image.
- **Middle Layers (Corners & Textures):** Combine edges to recognize corners, textures, and repeated patterns.
- **Deep Layers (Object Parts):** Learn complex shapes and meaningful object parts by combining lower-level features.
- **Final Layers (Complete Object Representation):** Integrate all extracted features to form a complete representation of the object and perform image classification.

Colab Notebook Link: <https://shorturl.at/0nS6x>